Survival Analysis

General Principles

Survival analysis studies the time until an event of interest (e.g., death, recovery, information acquisition) occurs. When analyzing binary survival outcomes (e.g., alive or dead), we can use models such as Cox proportional hazards to evaluate the effect of predictors on survival probabilities.

Key concepts include:

1. **Hazard Function**: The instantaneous risk of the event occurring at a given time.
2. **Survival Function**: The probability of surviving until a given time.
3. **Covariates**: Variables (e.g., age, treatment) that may affect survival probabilities.
4. **Baseline Hazard**: The hazard when all covariates are zero, which forms the reference for comparing different conditions.

Considerations

> **Note**
>
> - In survival analysis:
>   - The **baseline hazard** can follow distributions like Exponential, Weibull, or Gompertz, depending on the data.
>   - Censoring (when the event is not observed for some subjects) must be accounted for in the likelihood function. Proper handling is essential for unbiased results.
> - Bayesian survival models allow flexible handling of time-dependent covariates, random effects, and incorporate uncertainty more naturally than Frequentist methods.

Example

Here's an example of a Bayesian survival analysis using the **BayesForge (BF)** package. The data come from a clinical trial of mastectomy for breast cancer. The goal is to estimate the effect of the `metastasized` covariate, coded as 0 (no metastasis) and 1 (metastasis), on the survival outcome `event` for each patient. Time is continuous and censoring is indicated by the event variable.

Python

Data and discrete-time setup (shared by both approaches below).

```
from BayesForge import bf
import jax.numpy as jnp

m = bf(platform='cpu')

# --- Data ------------------------------------------------------------
# Mastectomy trial. Columns: `time` = follow-up length, `event` = 1 if
# death was observed, 0 if the patient was censored (left the study
# event-free), `metastasized` = "yes"/"no".
data_path = m.load.mastectomy(only_path=True)
m.data(data_path, sep=',')
m.df.metastasized = jnp.asarray(m.df.metastasized == "yes", dtype=jnp.int64)

# --- Piecewise-constant hazard setup ---------------------------------
# Cut follow-up time into fixed intervals of length 3. For every
# (subject, interval) this builds two matrices:
#   death[i, k]    -> 1 if subject i had the event in interval k
#   exposure[i, k] -> time subject i spent at risk in interval k
# Censoring is encoded here: after a subject leaves (event OR censoring)
# its exposure is 0 in every later interval, so it stops contributing
# to the likelihood without being counted as a death.
m.models.survival.import_time_even(jnp.asarray(m.df.time.values),
                                   jnp.asarray(m.df.event.values),
                                   interval_length=3)

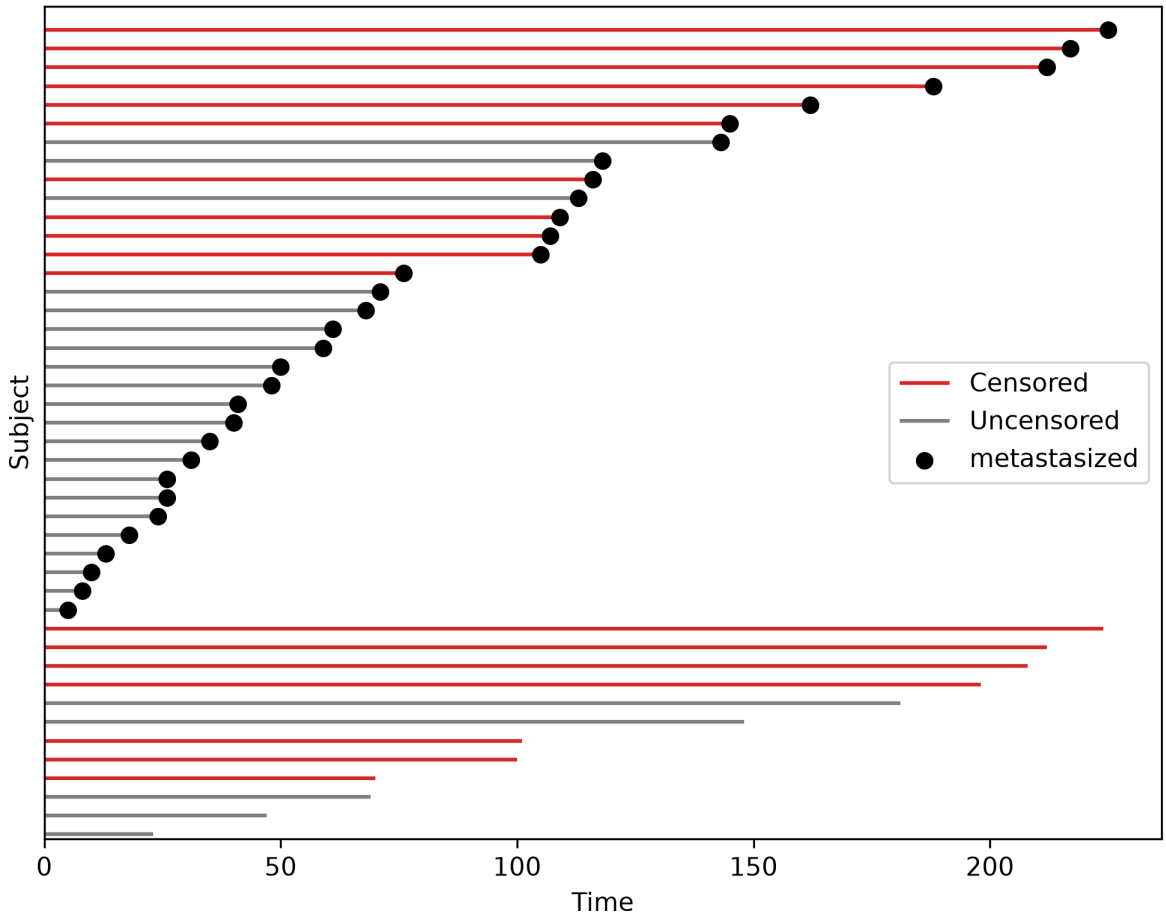
# Register `metastasized` as a patient-level (time-invariant) covariate.
m.models.survival.import_covF(jnp.asarray(m.df.metastasized.values), ["metastasized"])

# --- Censoring plot --------------------------------------------------
# One horizontal line per subject = observed follow-up. Red = censored
```

```
# (event never seen), grey = event observed, black dot = metastasized.
# Reading it: metastasized patients tend to have shorter lines ending
# in an event -> a first visual clue of a positive hazard effect.
m.models.survival.plot_censoring(cov="metastasized")
```

```
An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 16
--------------------------------------------------------------------------------
Survival concern 44 individuals in 76 intervals.
26.0 individuals experienced the event.
--------------------------------------------------------------------------------
Covariates imported: ['metastasized']
Surv object now has 1 covariates: ['metastasized']
```



Explicit model

The full model written out, to show every prior and the likelihood.

```
def model(death, cov, exposure, censoring=None):
    # --- Priors ------------------------------------------------
    # Baseline hazard: ONE positive rate per time interval. A
    # Gamma(0.01, 0.01) prior is almost flat on (0, inf), so the shape
    # of the hazard over time is driven by the data, not by a
    # parametric family (Weibull, Gompertz, ...). This is what makes
    # the model semi-parametric.
    lambda0 = m.dist.gamma(
      0.01, 0.01, shape=(exposure.shape[1],),
      name="Baseline_rate")

    # Covariate effect on the LOG hazard. Normal(0, 10) is weakly
    # informative: centred on 0 (no effect) but wide enough to let the
    # data move it. exp(beta) is the hazard ratio for metastasized.
    beta = m.dist.normal(
      0, 10, shape=(1,),
      name="Hazard_rate_metastasized")

    # --- Likelihood --------------------------------------------
    # Proportional hazards: lambda[i,k] = lambda0[k] * exp(beta * x_i).
    # Expected event count mu = lambda * exposure; the 0/1 event
    # indicator is modelled as Poisson (equivalent to the piecewise
    # exponential survival likelihood).
    lambda_ = jnp.exp(beta * cov.squeeze())[:, None] * lambda0[None, :]
    mu = exposure * lambda_
    m.dist.poisson(mu + jnp.finfo(mu.dtype).tiny, obs=death)

# 4 chains and target_accept_prob=0.99: the many near-empty late
# baseline intervals make the geometry awkward, so a higher acceptance
# target keeps the step size small enough to mix well.
m.fit(model, num_samples=2000, num_warmup=2000, num_chains=4,
      target_accept_prob=0.99, progress_bar=False)
print(m.summary())

# Posterior cumulative hazard and survival curves, by covariate group.
m.models.survival.plot_surv(beta="Hazard_rate_metastasized")
```

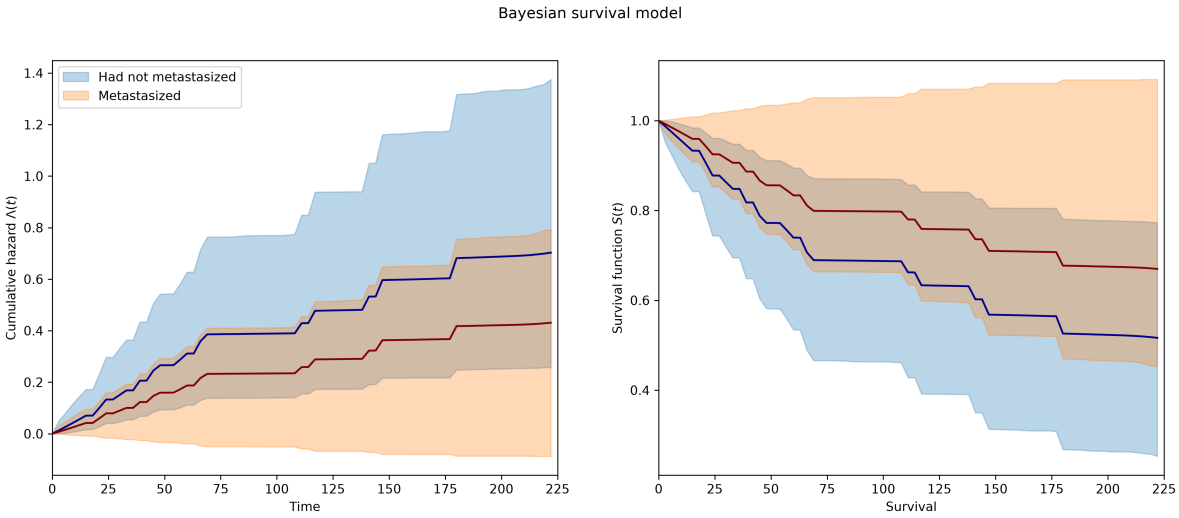
```
                          mean     sd  eti89_lb  eti89_ub  ess_bulk  \
Baseline_rate[0]          0.00   0.00      0.00      0.00   2127.78
```

```
Baseline_rate[1]              0.00   0.01      0.00      0.01   3305.32
Baseline_rate[2]              0.00   0.01      0.00      0.01   3309.77
Baseline_rate[3]              0.00   0.01      0.00      0.01   2971.83
Baseline_rate[4]              0.00   0.01      0.00      0.01   3554.17
...                            ...    ...       ...       ...       ...
Baseline_rate[72]             0.00   0.01      0.00      0.00   2034.68
Baseline_rate[73]             0.00   0.01      0.00      0.00   1856.24
Baseline_rate[74]             0.00   0.01      0.00      0.00   2011.07
Baseline_rate[75]             1.18  11.59      0.00      0.20   2056.81
Hazard_rate_metastasized[0]   0.78   0.49      0.04      1.61   1577.74

                            ess_tail  r_hat  mcse_mean  mcse_sd
Baseline_rate[0]             2290.04    1.0       0.00     0.00
Baseline_rate[1]             2868.80    1.0       0.00     0.00
Baseline_rate[2]             2987.97    1.0       0.00     0.00
Baseline_rate[3]             2879.83    1.0       0.00     0.00
Baseline_rate[4]             3204.71    1.0       0.00     0.00
...                              ...    ...        ...      ...
Baseline_rate[72]            2160.30    1.0       0.00     0.00
Baseline_rate[73]            1809.47    1.0       0.00     0.00
Baseline_rate[74]            2331.83    1.0       0.00     0.00
Baseline_rate[75]            2139.28    1.0       0.13     1.22
Hazard_rate_metastasized[0]  1934.01    1.0       0.01     0.01

[77 rows x 9 columns]

(<Figure size 4800x1800 with 2 Axes>,
 (<Axes: xlabel='Time', ylabel='Cumulative hazard $\\Lambda(t)$'>,
  <Axes: xlabel='Survival', ylabel='Survival function $S(t)$'>))
```



Built-in model

Same model, assembled by BF from the imported survival object. Priors are attributes you can override before `fit`.

```
# Optional: change the default priors (shown with their defaults).
m.models.survival.baseline_rate_prior = (0.01, 0.01)   # Gamma(conc, rate)
m.models.survival.hazard_rate_prior_scale = 10.0        # Normal(0, scale)

m.fit(m.models.survival.model)
print(m.summary())

m.models.survival.plot_surv(beta="Hazard_rate_metastasized")
```

```
This function is still in development. Use it with caution.

  0%|          | 0/2000 [00:00<?, ?it/s]

  0%|          | 0/2000 [00:00<?, ?it/s]

  0%|          | 0/2000 [00:00<?, ?it/s]

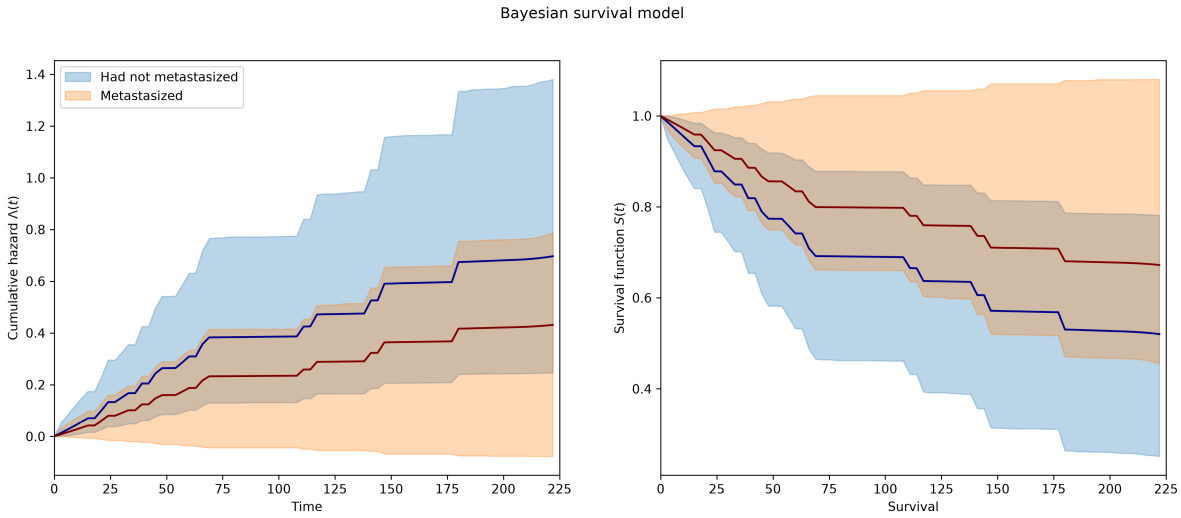
  0%|          | 0/2000 [00:00<?, ?it/s]
```

```
                         mean      sd  eti89_lb  eti89_ub  ess_bulk  ess_tail  \
Baseline_rate[0]         0.00    0.00      0.00      0.00   1116.36   1156.07
Baseline_rate[1]         0.00    0.01      0.00      0.01   1878.03   1671.16
Baseline_rate[2]         0.00    0.01      0.00      0.01   2041.70   1737.83
Baseline_rate[3]         0.00    0.01      0.00      0.01   1538.22   1567.86
Baseline_rate[4]         0.00    0.01      0.00      0.01   1725.54   1366.50
...                       ...     ...       ...       ...       ...       ...
log_Baseline_rate[71] -104.03  103.08   -296.36     -8.78   1009.60   1162.48
log_Baseline_rate[72] -105.32  100.98   -298.76     -9.13    957.54    964.18
log_Baseline_rate[73] -102.76   99.48   -293.32     -8.71    872.55    914.55
log_Baseline_rate[74]  -97.55   98.11   -282.91     -8.07    941.97    990.71
log_Baseline_rate[75] -103.36  107.20   -323.23     -1.39   1039.31   1029.70

                       r_hat  mcse_mean  mcse_sd
Baseline_rate[0]         1.0       0.00     0.00
Baseline_rate[1]         1.0       0.00     0.00
Baseline_rate[2]         1.0       0.00     0.00
Baseline_rate[3]         1.0       0.00     0.00
Baseline_rate[4]         1.0       0.00     0.00
...                      ...        ...      ...
log_Baseline_rate[71]    1.0       3.69     6.01
log_Baseline_rate[72]    1.0       3.51     4.23
log_Baseline_rate[73]    1.0       3.84     5.44
log_Baseline_rate[74]    1.0       3.78     7.47
log_Baseline_rate[75]    1.0       3.77     4.26

[153 rows x 9 columns]
```

```
(<Figure size 4800x1800 with 2 Axes>,
 (<Axes: xlabel='Time', ylabel='Cumulative hazard $\\Lambda(t)$'>,
  <Axes: xlabel='Survival', ylabel='Survival function $S(t)$'>))
```



Survival plot. Left: posterior cumulative hazard $\Lambda(t) = \sum_k \lambda_0^{(k)} \Delta_k$ per covariate group; right: the implied survival $S(t) = e^{-\Lambda(t)}$. Solid line = posterior mean, shaded band = 94% credible interval, blue = not metastasized, orange = metastasized. The size and direction of the effect are read from β (equivalently the hazard ratio e^{β}) in `m.summary()`; here $\beta > 0$, i.e. metastasis raises the hazard.

R

Data and discrete-time setup (shared by both approaches below).

```r
library(BayesForge)
m <- importBF(platform = "cpu")
jnp <- reticulate::import("jax.numpy")

# --- Data ------------------------------------------------------------
# Mastectomy trial. Columns: `time` = follow-up length, `event` = 1 if
# death was observed, 0 if the patient was censored (left the study
# event-free), `metastasized` = "yes"/"no".
data_path <- m$load$mastectomy(only_path = TRUE)
m$data(data_path, sep = ",")
met <- jnp$asarray(m$df$metastasized == "yes", dtype = jnp$int64)
m$df$metastasized <- met

# --- Piecewise-constant hazard setup ---------------------------------
# Cut follow-up time into fixed intervals of length 3. For every
# (subject, interval) this builds two matrices:
```

```
#   death[i, k]    -> 1 if subject i had the event in interval k
#   exposure[i, k] -> time subject i spent at risk in interval k
# Censoring is encoded here: after a subject leaves (event OR censoring)
# its exposure is 0 in every later interval, so it stops contributing
# to the likelihood without being counted as a death.
m$models$survival$import_time_even(jnp$asarray(m$df$time),
                                   jnp$asarray(m$df$event),
                                   interval_length = 3L)

# Register `metastasized` as a patient-level (time-invariant) covariate.
m$models$survival$import_covF(met, list("metastasized"))

# --- Censoring plot ------------------------------------------------
# One horizontal line per subject = observed follow-up. Red = censored
# (event never seen), grey = event observed, black dot = metastasized.
# Reading it: metastasized patients tend to have shorter lines ending
# in an event -> a first visual clue of a positive hazard effect.
m$models$survival$plot_censoring(cov = "metastasized")
```

Explicit model

The full model written out, to show every prior and the likelihood.

```
model <- function(death, cov, exposure, censoring = NULL) {
  # --- Priors -------------------------------------------------
  # Baseline hazard: ONE positive rate per time interval. A
  # Gamma(0.01, 0.01) prior is almost flat on (0, inf), so the shape
  # of the hazard over time is driven by the data, not by a
  # parametric family (Weibull, Gompertz, ...). This is what makes
  # the model semi-parametric.
  lambda0 <- m$dist$gamma(
    0.01, 0.01, shape = reticulate::tuple(dim(exposure)[2]),
    name = "Baseline_rate")

  # Covariate effect on the LOG hazard. Normal(0, 10) is weakly
  # informative: centred on 0 (no effect) but wide enough to let the
  # data move it. exp(beta) is the hazard ratio for metastasized.
  beta <- m$dist$normal(
    0, 10, shape = reticulate::tuple(1L),
    name = "Hazard_rate_metastasized")

  # --- Likelihood ---------------------------------------------
```

```
  # Proportional hazards: lambda[i,k] = lambda0[k] * exp(beta * x_i).
  # Expected event count mu = lambda * exposure; the 0/1 event
  # indicator is modelled as Poisson (equivalent to the piecewise
  # exponential survival likelihood).
  lambda_ <- jnp$expand_dims(jnp$exp(beta * jnp$squeeze(cov)), 1L) *
    jnp$expand_dims(lambda0, 0L)
  mu <- jnp$multiply(exposure, lambda_)
  m$dist$poisson(mu + jnp$finfo(mu$dtype)$tiny, obs = death)
}

# 4 chains and target_accept_prob=0.99: the many near-empty late
# baseline intervals make the geometry awkward, so a higher acceptance
# target keeps the step size small enough to mix well.
m$fit(model, num_samples = 2000L, num_warmup = 2000L, num_chains = 4L,
      target_accept_prob = 0.99, progress_bar = FALSE)
print(m$summary())

# Posterior cumulative hazard and survival curves, by covariate group.
m$models$survival$plot_surv(beta = "Hazard_rate_metastasized")
```

Built-in model

Same model, assembled by BF from the imported survival object. Priors are attributes you can override before `fit`.

```
# Optional: change the default priors (shown with their defaults).
m$models$survival$baseline_rate_prior <- c(0.01, 0.01)  # Gamma(conc, rate)
m$models$survival$hazard_rate_prior_scale <- 10.0        # Normal(0, scale)

m$fit(m$models$survival$model)
print(m$summary())

m$models$survival$plot_surv(beta = "Hazard_rate_metastasized")
```

Julia

Data and discrete-time setup (shared by both approaches below).

```
using PyBayesForge
using PythonCall
m = importBF(platform="cpu")
```

```
# --- Data ------------------------------------------------------------
# Mastectomy trial. Columns: `time` = follow-up length, `event` = 1 if
# death was observed, 0 if the patient was censored (left the study
# event-free), `metastasized` = "yes"/"no".
data_path = pyconvert(String, m.load.mastectomy(only_path=true))
m.data(data_path, sep=",")
m.df.metastasized = jnp.asarray(m.df.metastasized.eq("yes").values, dtype=jnp.int64)

# --- Piecewise-constant hazard setup ---------------------------------
# Cut follow-up time into fixed intervals of length 3. For every
# (subject, interval) this builds two matrices:
#   death[i, k]    -> 1 if subject i had the event in interval k
#   exposure[i, k] -> time subject i spent at risk in interval k
# Censoring is encoded here: after a subject leaves (event OR censoring)
# its exposure is 0 in every later interval, so it stops contributing
# to the likelihood without being counted as a death.
m.models.survival.import_time_even(jnp.asarray(m.df.time.values),
                                   jnp.asarray(m.df.event.values),
                                   interval_length=3)

# Register `metastasized` as a patient-level (time-invariant) covariate.
m.models.survival.import_covF(jnp.asarray(m.df.metastasized.values), ["metastasized"])

# --- Censoring plot --------------------------------------------------
# One horizontal line per subject = observed follow-up. Red = censored
# (event never seen), grey = event observed, black dot = metastasized.
# Reading it: metastasized patients tend to have shorter lines ending
# in an event -> a first visual clue of a positive hazard effect.
@pyplot m.models.survival.plot_censoring(cov="metastasized")
```

Explicit model

The full model written out, to show every prior and the likelihood.

```
@BF function model(death, cov, exposure)
    # --- Priors ------------------------------------------------------
    # Baseline hazard: ONE positive rate per time interval. A
    # Gamma(0.01, 0.01) prior is almost flat on (0, inf), so the shape
    # of the hazard over time is driven by the data, not by a
    # parametric family (Weibull, Gompertz, ...). This is what makes
    # the model semi-parametric.
```

```
    lambda0 = m.dist.gamma(
      0.01, 0.01, shape = (pyconvert(Int, exposure.shape[1]),),
      name = "Baseline_rate")

    # Covariate effect on the LOG hazard. Normal(0, 10) is weakly
    # informative: centred on 0 (no effect) but wide enough to let the
    # data move it. exp(beta) is the hazard ratio for metastasized.
    beta = m.dist.normal(
      0, 10, shape = (1,),
      name = "Hazard_rate_metastasized")

    # --- Likelihood -------------------------------------------------
    # Proportional hazards: lambda[i,k] = lambda0[k] * exp(beta * x_i).
    # Expected event count mu = lambda * exposure; the 0/1 event
    # indicator is modelled as Poisson (equivalent to the piecewise
    # exponential survival likelihood).
    lambda_ = jnp.expand_dims(jnp.exp(beta * jnp.squeeze(cov)), 1) *
      jnp.expand_dims(lambda0, 0)
    mu = exposure * lambda_
    m.dist.poisson(mu + jnp.finfo(mu.dtype).tiny, obs = death)
end

# 4 chains and target_accept_prob=0.99: the many near-empty late
# baseline intervals make the geometry awkward, so a higher acceptance
# target keeps the step size small enough to mix well.
m.fit(model, num_samples = 2000, num_warmup = 2000, num_chains = 4,
      target_accept_prob = 0.99, progress_bar = false)
println(m.summary())

# Posterior cumulative hazard and survival curves, by covariate group.
@pyplot m.models.survival.plot_surv(beta="Hazard_rate_metastasized")
```

Built-in model

Same model, assembled by BF from the imported survival object. Priors are attributes you can override before `fit`.

```
# Optional: change the default priors (shown with their defaults).
m.models.survival.baseline_rate_prior = (0.01, 0.01)   # Gamma(conc, rate)
m.models.survival.hazard_rate_prior_scale = 10.0        # Normal(0, scale)

m.fit(m.models.survival.model)
```

```
println(m.summary())

@pyplot m.models.survival.plot_surv(beta="Hazard_rate_metastasized")
```

Mathematical Details

Bayesian formulation

The BF survival model uses a **piecewise-constant hazard** (Poisson counting-process) approach. The continuous follow-up time is divided into K fixed intervals of length Δ. Each subject i in interval k contributes:

$$N_{ik} \sim \text{Poisson}(\mu_{ik})$$

$$\mu_{ik} = \lambda_{ik} \cdot e_{ik}$$

$$\lambda_{ik} = \lambda_0^{(k)} \exp(\beta X_i)$$

$$\lambda_0^{(k)} \sim \text{Gamma}(0.01,\ 0.01)$$

$$\beta \sim \text{Normal}(0,\ 10)$$

Where:

- N_{ik} is the number of events (0 or 1) recorded for subject i in interval k.
- e_{ik} is the **exposure** (time at risk) for subject i in interval k. It equals Δ if the subject is fully observed in the interval, a fractional value if the subject exits (via event or censoring) mid-interval, and 0 if the subject has already left the risk set. This term is the mechanism that handles censoring: once a subject is censored or experiences the event, their exposure drops to 0 in all subsequent intervals.
- $\mu_{ik} = \lambda_{ik} \cdot e_{ik}$ is the expected number of events, i.e., the hazard rate scaled by time at risk.
- λ_{ik} is the hazard rate for subject i in interval k, decomposed into a **baseline hazard** $\lambda_0^{(k)}$ and a covariate-specific multiplicative shift.

- $\lambda_0^{(k)}$ is the **baseline hazard** in interval k, constant within each interval but free to vary across intervals. This gives the model non-parametric flexibility over time. A Gamma$(0.01, 0.01)$ prior (BF default `m.models.survival.baseline_rate_prior`) places minimal information on the baseline hazard; on the numpyro backend it is sampled in log space for stability.
- X_i is the vector of covariates for subject i.
- β is the vector of regression coefficients. The coefficient $\exp(\beta)$ gives the **hazard ratio**: the multiplicative change in hazard for a unit increase in the corresponding covariate. A weakly informative Normal$(0, 10)$ prior is centred on "no effect" but wide on the log-hazard scale. It is the BF default (`m.models.survival.hazard_rate_prior_scale`).

Survival and hazard functions

From the posterior samples of $\lambda_0^{(k)}$ and β, two key quantities can be derived:

Cumulative hazard up to interval K:

$$\Lambda_i(t_K) = \sum_{k=1}^{K} \lambda_0^{(k)} \exp(\beta X_i)\, \Delta_k$$

Survival function:

$$S_i(t_K) = \exp(-\Lambda_i(t_K))$$

Where Δ_k is the width of interval k.

Model validation tests

- Comparison with PyMC: `Survival_analysis_verify.py` fits the mastectomy survival model with BayesForge and PyMC and checks that the baseline rates and the hazard ratio agree.
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Reference(s)

https://en.wikipedia.org/wiki/Proportional_hazards_model https://www.mathworks.com/help/stats/cox-proportional-hazard-regression.html https://www.pymc.io/projects/examples/en/latest/survival_analysis/surv https://vflores-io.github.io/posts/20240924_numpyro_logreg_surv_analysis/np01_logreg_surv_analysis/